Devoid Viewer

Codebase & Installer Documentation

This document explains every file, struct, method, and design decision in the Devoid Viewer codebase, and documents the WiX installer. It is written for someone new to Rust who wants to understand not only *what* the code does but *why* it was written that way.

Table of Contents

Section	Topic	Page
1	Project structure overview	2
2	Cargo.toml — dependencies explained	2
3	nav.rs — directory navigation	4
4	loader.rs — image decoding	6
5	main.rs — application core	8
6	Threading & asynchronous design	17
7	Key design decisions & trade-offs	19
8	Caveats & known limitations	21
9	Installer — main.wxs	21
10	Glossary of Rust concepts used	24

1. Project Structure Overview

The project follows the standard Cargo layout:

File	Responsibility
`Cargo.toml`	Project name, version, and all external dependencies.
`build.rs`	Build script — embeds devoid.ico and version metadata into the exe via winres.
`src/main.rs`	Event loop, app state, input handling, pixel renderer, window management.
`src/loader.rs`	Image decoding for all supported formats. Owns Frame and DecodedImage.
`src/nav.rs`	Directory scanning, human-sorted sibling list, next/prev navigation.
`wix/main.wxs`	WiX installer definition — features, file associations, shortcuts.
`devoid.ico`	Multi-size application icon (16, 32, 48, 256 px). Used by the exe, title bar, taskbar, and installer.

2. Cargo.toml — Dependencies Explained

Each crate was chosen for leanness. The dependency graph is roughly 50–70 crates and the release binary around 2–3 MB, compared to 200+ crates and ~9 MB with a full GUI toolkit.

Windowing: winit

winit is the de-facto standard Rust crate for OS windows and input events. It handles the event loop, keyboard, window resize, drag-and-drop, and monitor queries. The rwh_05 feature enables the raw-window-handle v0.5 interface that softbuffer requires. On Windows the platform extension EventLoopBuilderExtWindows is used to install a Win32 message hook.

Pixel display: softbuffer

softbuffer provides a Vec<u32> that maps directly to the window's pixels. No GPU pipeline, no shader compilation. On Windows it uses a GDI DIB section, compositor-accelerated without requiring a GPU context.

> ■ *DWM fills every new window with white before the application paints. Mitigated by hiding the window until the image is ready, and by painting gray synchronously before the event loop starts. The white background is never visible in normal use.*

Native dialogs: windows-sys

windows-sys is a zero-overhead Windows API binding. Used for MessageBoxW (delete and error dialogs), SystemParametersInfoW(SPI_GETWORKAREA) (precise taskbar exclusion), and PostMessageW (taskbar thumbnail close fix). Target-specific — does not affect non-Windows builds.

Image decoding: image

The **image** crate handles BMP, PNG, JPEG, GIF, WebP, TIFF, and ICO. Features enabled selectively. GIF animation uses image::codecs::gif::GifDecoder with AnimationDecoder. The ICO format is also used at startup to decode devoid.ico for the window icon.

JPEG XL: jxl-oxide

Pure-Rust JPEG XL decoder. Not yet part of the image crate. Uses the low-level streaming API.

SVG: resvg + usvg + tiny-skia

- **usvg** parses and normalises the SVG scene tree.
- **resvg** renders into a tiny-skia Pixmap.
- **tiny-skia** rasterises to premultiplied RGBA, which is then unpremultiplied to straight RGBA.

Channel communication: crossbeam-channel

Faster and more ergonomic than std::sync::mpsc. Bounded at capacity 32 to throttle background threads if the UI falls behind.

Windows resource embedding: winres (build dependency)

Runs at build time only. Embeds devoid.ico and version metadata into the exe. Zero runtime cost.

3. nav.rs — Directory Navigation

Maintains a human-sorted list of sibling image files in the same directory as the currently open file. Navigation wraps at both ends.

struct DirNavigator

- **files: Vec<PathBuf>** — human-sorted sibling list.
- **index: usize** — position of the currently displayed file.

Derives **Default** — auto-generated empty initialiser, so DirNavigator::default() creates an empty navigator with no allocation.

fn set_current(path)

Called on every image open. Compares the new path's parent directory against the first entry in the current list to decide whether to rebuild. If the directory is the same, only the index is updated. Falls back to a single-entry list if the path is not found in the scan — this handles the case where a file was opened from outside the scanned directory.

fn rebuild(dir)

Reads the directory with std::fs::read_dir, filters for files with supported extensions, and sorts using **human_sort** so that img2 sorts before img10.

fn next() / fn prev()

Both return None when fewer than 2 images exist — navigation on a single-image directory is a silent no-op, preventing a redundant reload of the same image. Both wrap around using modulo and checked_sub for safe unsigned arithmetic.

fn peek_next() / fn peek_prev()

Read-only lookahead used by preload_adjacent(). Also returns None for single-image directories so no unnecessary decode threads are spawned.

fn remove(path)

Removes a path after deletion. Uses saturating_sub to keep the index in bounds when the deleted file was the last one.

fn current()

Returns the currently active path, which may differ from the last set_current argument if remove was called since then.

fn human_sort / fn consume_number

Splits each filename into alternating text and digit runs. Digit runs are parsed as u64 and compared numerically, so *img2.png* sorts before *img10.png*. saturating_mul and saturating_add prevent overflow on pathologically long digit sequences.

4. loader.rs — Image Decoding

Owns the image type definitions and all decoding logic. Returns **DecodedImage** directly — no conversion needed in main.rs.

struct Frame

pixels: Vec<u8> — straight RGBA8, row-major. **delay_ms: u32** — frame duration. Zero for non-animated formats.

struct DecodedImage

One or more frames sharing the same canvas dimensions. **is_animated()** returns true when more than one frame exists. **frame(idx)** borrows a specific frame. **single()** is a private constructor for single-frame images.

fn load_image(path) → LoadResult

Dispatches by lowercased file extension: svg → load_svg, jxl → load_jxl, gif → load_gif, everything else → load_standard.

fn load_standard — BMP, PNG, JPEG, WebP, TIFF, ICO

image::open() decodes to a DynamicImage, which is then normalised to RGBA8 via into_rgba8().

fn load_gif — Animated GIF

Uses GifDecoder with AnimationDecoder. Image dimensions are read before the frames are consumed — a single file open. Per-frame delay comes from frame.delay().numer_denom_ms(), with a 20ms minimum to prevent zero-delay GIFs from locking up the event loop, and a 100ms fallback when the denominator is zero.

fn load_jxl — JPEG XL

- A BufReader wraps the file for efficient I/O.
- request_color_encoding(srgb(...)) forces sRGB output regardless of the file's embedded colour space.
- render_frame(0) decodes the first frame.
- The stream yields interleaved f32 values. Channels 1, 2, 3, and 4 are each handled explicitly to produce 4-channel RGBA u8 output via f32_to_u8, which clamps to [0,1] then scales to 255.

fn load_svg — SVG

- usvg parses and normalises the scene tree.
- resvg renders to a tiny-skia Pixmap (premultiplied RGBA).
- Each pixel is unpremultiplied by dividing R, G, B by alpha. Fully transparent pixels are zeroed.

5. main.rs — Application Core

Contains the winit event loop, all application state, and the pixel renderer. No widget toolkit, no font renderer, no GPU pipeline.

5.1 const BG: u8 = 42

Background grey used in the blank state, alpha compositing, and the pre-loop synchronous surface fill. All three call pack_rgb(BG, BG, BG) so the colour is defined once.

5.2 enum Msg

Four variants sent from background threads to the UI thread via the crossbeam channel:

- **Loaded** — primary decode finished. Carries the path, decoded image, and OpenMode so the UI knows whether to resize the window.
- **Preloaded** — speculative decode for the neighbour cache.
- **Error** — decode failed. Carries the path and error string.
- **Deleted** — async deletion result: Ok(()) or Err(String).

5.3 struct WakeUp / type Proxy

WakeUp is a zero-size marker type. Background threads inject one via proxy.send_event(WakeUp) to wake the sleeping event loop after posting a Msg. **Proxy** is a type alias that shortens the verbose EventLoopProxy<WakeUp> throughout the file.

5.4 struct View<'a>

A lightweight snapshot passed to render() each frame. Holds a borrowed reference to the current DecodedImage and the active frame index. The lifetime 'a ensures the view cannot outlive the App that owns the image. Keeping this separate from App means the renderer has no access to mutable state.

5.5 enum OpenMode

FirstImage — the window resizes to fit the image. Used on the first open. **Navigate** — the window stays the same size. Used for all subsequent navigation and drag-and-drop. Carried directly through the Msg::Loaded variant so no bool-to-enum conversion is needed in drain_messages.

5.6 struct App — fields

- **current_path / current_image** — the displayed file. current_image is not cleared when navigation starts; the previous image stays on screen until the new one is ready.
- **frame_index / next_frame_at** — GIF animation state. next_frame_at is None for non-animated images.
- **navigator** — the DirNavigator for the current directory.
- **msg_tx / msg_rx** — the crossbeam channel endpoints.

- **loading_path** — the path currently being decoded. Used to discard stale Loaded messages if the user navigated away before the decode finished.
- **preload_cache** — decoded images for the neighbouring files, keyed by path.
- **deleting** — prevents a second Delete keypress while an async deletion is in flight.
- **pending_center** — set when a resize is issued; cleared on the next WakeUp when the resize has landed and center_window() can be called accurately.

5.7 fn open_image()

Cache hit: calls apply_image directly without spawning a thread. Cache miss: seeds the navigator immediately so peek_next/peek_prev are accurate for preloading, sets loading_path, then spawns a background thread that calls load_image, posts a Msg::Loaded or Msg::Error, and fires WakeUp.

5.8 fn apply_image() → Option<(u32, u32)>

Stores the image and path, clears loading_path, updates the navigator, starts preloading adjacent files, and resets the animation. Returns the image dimensions when OpenMode::FirstImage — a return value rather than a side-effect field, which keeps the caller in control of when to issue the resize.

5.9 fn clear_image()

Sets both current_image and current_path to None, then resets the animation state. Called after the last image in a directory is deleted.

5.10 fn preload_adjacent()

Spawns up to two background decode threads — one for the next sibling and one for the previous. Both peek_next and peek_prev return None for single-image directories, so no threads are spawned unnecessarily. Already-cached paths are skipped.

5.11 fn reset_animation() / fn tick_animation()

reset_animation() sets frame_index to 0 and computes the first frame deadline for animated images. For non-animated images next_frame_at stays None so the event loop sleeps indefinitely between redraws.

tick_animation() checks whether the current deadline has passed, advances the frame index, and schedules the next deadline. Returns true when a redraw is needed. Uses an if-let early return rather than unwrap().

5.12 fn navigate(forward: bool)

Calls navigator.next() or navigator.prev(). Both return None for single-image directories, making navigation a silent no-op rather than reloading the same file.

5.13 fn request_delete() / fn on_deleted()

request_delete() sets the deleting flag and spawns a thread that calls std::fs::remove_file. The UI never blocks on I/O. The result arrives as Msg::Deleted.

on_deleted() clears the flag, removes the path from the navigator, clears the preload cache (which may contain the now-deleted file), then either navigates to the next sibling or calls clear_image() if the directory is now empty.

5.14 fn drain_messages() → Option<(u32, u32)>

A non-blocking try_recv loop that processes all pending messages. Loaded messages whose path does not match loading_path are silently discarded — this handles the race where the user navigates faster than images decode. Returns the resize dimensions as a value rather than mutating a field.

5.15 Rendering functions

- **fit_scale()** — computes the largest uniform scale that fits the image within the window. Takes all four dimensions as u32 and converts to f32 internally.
- **nearest()** — point-samples an RGBA8 pixel buffer at float source coordinates by truncating to the nearest integer. Clamps to the image boundary.
- **composite()** — alpha-blends a straight-RGBA pixel over the background colour BG. The inner blend() function is shared across the R, G, and B channels.
- **pack_rgb()** — packs three u8 values into softbuffer's 0x00RRGGBB u32 format.
- **render()** — iterates every window pixel as a double for-loop (row then column). dx and dy are cheap loop counters, not computed from a flat index. Always paints unconditionally so that off-screen regions are correct when the window is dragged back on-screen.

5.16 fn to_wide() / fn confirm_delete_dialog() / fn show_error_dialog()

to_wide() encodes a Rust &str as a null-terminated UTF-16 Vec, the format required by Win32 API calls. Defined once, used in both dialog functions.

Both dialogs call **MessageBoxW** which is blocking — correct for a modal confirmation. #[cfg(not(windows))] fallbacks allow the code to compile on other platforms.

5.17 Window helpers

work_area() returns the usable desktop area in logical pixels. On Windows calls SPI_GETWORKAREA for the exact taskbar-excluded rectangle via windows_work_area(). Elsewhere uses 90% × 85% of the monitor size as a heuristic.

clamp_to_work_area() scales image dimensions down to fit the usable area, never up. **center_window()** positions the window at the monitor centre with a minimum Y offset of 30 × scale_factor pixels so the title bar is always reachable.

5.18 fn load_window_icon()

Embeds devoid.ico at compile time via include_bytes!("../devoid.ico"). The image crate decodes it and the best-fitting frame is passed to winit as RGBA bytes. Set on the WindowBuilder via with_window_icon(), which sets both the title bar and taskbar button icons.

■ *devoid.ico contains four sizes: 16×16, 32×32, 48×48, and 256×256 — generated from the original 256×256 source with Lanczos downscaling. Windows picks the closest size for each context rather than scaling a single large image.*

5.19 fn image_size_hint()

Reads only the file header to get image dimensions — no full decode. SVG is the exception: the entire document must be parsed since there is no header-only path. Used at startup to size the window correctly before the background decode thread finishes.

5.20 fn main() — startup sequence

- **Header probe** — image_size_hint() gives dimensions without a full decode. The window appears at the correct size immediately.
- **EventLoopBuilder type annotation** — let mut el_builder: EventLoopBuilder<WakeUp> avoids a turbofish in the expression.
- **WM_SYSCOMMAND/SC_CLOSE hook** — the taskbar thumbnail close button sends this message, which winit 0.29 does not translate into CloseRequested. The msg_hook intercepts it and re-posts WM_CLOSE, which winit handles correctly. Both close paths reach the same CloseRequested handler.
- **with_visible(!start_hidden)** — the window is hidden when an image argument is provided. Revealed only after resize and centering are applied, so the user sees the window appear already showing the image at the correct position.
- **Synchronous gray paint** — the softbuffer surface is filled with BG gray and presented before the event loop starts, overwriting the DWM white background.
- **RedrawRequested always paints** — no dirty flag. Prevents stale pixels in regions that were off-screen and then exposed.
- **GIF control flow** — ControlFlow::WaitUntil at the next frame deadline when animating; ControlFlow::Wait otherwise. Zero CPU when idle.

6. Threading & Asynchronous Design

6.1 Why threads?

Image decoding is I/O-bound and CPU-bound. A large file can take hundreds of milliseconds. Doing this on the event loop thread would freeze the window for the duration.

6.2 The channel pipeline

- One bounded crossbeam channel, capacity 32.
- msg_tx is cloned cheaply for each background thread.
- msg_rx is drained in drain_messages() on every WakeUp.
- Background threads post a Msg then call proxy.send_event(WakeUp) to wake the event loop.

■ *proxy.send_event() is thread-safe — EventLoopProxy is Arc-based internally.*

6.3 ControlFlow::Wait — zero idle CPU

The event loop sleeps until an event arrives. ControlFlow::WaitUntil wakes precisely at the next GIF frame deadline. When displaying a still image the process uses effectively zero CPU.

6.4 Header probe + async decode

image_size_hint() reads only the file header for dimensions. The window is created and shown at the correct size with a gray background. The full decode runs on a background thread. This keeps startup fast regardless of image file size.

6.5 Window reveal sequence

When an image argument is provided: window is hidden → gray surface paint → decode thread starts → WakeUp arrives → resize issued → pending_center = true → next WakeUp: center_window() → set_visible(true) → RedrawRequested paints the image. The user sees the window appear already showing the correct content at the correct size and position.

6.6 Taskbar thumbnail close button

The taskbar thumbnail close button sends WM_SYSCOMMAND/SC_CLOSE. Winit 0.29 handles WM_CLOSE (the window X button) but not WM_SYSCOMMAND/SC_CLOSE. The msg_hook re-posts it as WM_CLOSE so both paths reach the same CloseRequested handler.

6.7 Stale result discarding

Loaded messages are checked against loading_path. A mismatch means the user navigated away before the decode finished — the message is silently dropped and the correct image will arrive via its own thread.

6.8 Async deletion

remove_file() runs on a background thread. The deleting flag prevents a second deletion being requested while the first is in flight.

6.9 Off-screen pixel correctness

RedrawRequested always paints the full window. When the OS exposes a region that was clipped because the window was partially off-screen, the next redraw writes fresh pixels for the entire window rather than leaving stale content.

6.10 The two-event resize/centre sequence

window.request_inner_size() is asynchronous — the resize does not take effect immediately. Setting pending_center = true on the WakeUp that issues the resize, then calling center_window() on the next WakeUp, ensures the window is centred after its new size has actually been applied.

7. Key Design Decisions & Trade-offs

winit + softbuffer over a GUI toolkit

A full GUI toolkit such as egui adds ~200+ crates and ~9 MB to the binary. winit + softbuffer totals ~50–70 crates and ~2–3 MB. The trade-off is that native OS dialogs must be called directly for delete confirmation and error display.

Fit-to-window rendering

Every image is scaled to fit the current window using fit_scale(), which always maintains the aspect ratio. The window itself is the zoom control — resizing it rescales the image. This is simpler and more predictable than explicit zoom: every image is always fully visible, always correctly proportioned, and there is no hidden state to get confused by.

Nearest-neighbour sampling

The renderer uses point sampling — the source pixel at the integer-truncated coordinate. This is correct and fast at fit-to-window scale, where no magnification occurs. Bilinear filtering would add complexity for no visible benefit at this scale.

OpenMode carried through Msg::Loaded

The resize bool was replaced with mode: OpenMode in the Loaded variant. This removes the bool-to-enum conversion in drain_messages and makes the intent explicit at the point where the message is constructed.

Single-image navigation is a no-op

navigator.next() and navigator.prev() return None when fewer than 2 images exist. This prevents a redundant reload of the same image when navigating a single-image directory.

No dirty flag — always paint on RedrawRequested

A dirty flag would prevent unnecessary repaints but would also leave stale pixels in off-screen regions when the window is partially off-screen and then moved back. Always painting is correct, and with ControlFlow::Wait the event loop only delivers RedrawRequested when something has actually changed.

to_wide() helper for Win32 UTF-16 encoding

The OsStr::encode_wide().chain(Some(0)).collect() pattern would otherwise appear four times across the two dialog functions. One helper eliminates the repetition.

include_bytes! for the window icon

Embedding the ico at compile time means no runtime file path, no missing-file failure in production, and the icon is always available regardless of the working directory.

Multi-size ICO file

A single 256×256 image produces a blocky title bar icon because Windows scales it with nearest-neighbour to 16×16. The ico contains 16×16, 32×32, 48×48, and 256×256 frames — generated with Lanczos downscaling. Windows picks the closest size for each context.

WM_SYSCOMMAND/SC_CLOSE re-post

Re-posting WM_CLOSE from the msg_hook is the minimal correct fix for the taskbar thumbnail close bug. No new Msg variants, no shared state, no atomic flags — the existing CloseRequested handler does all the work.

Return values over side effects

apply_image() returns Option<(u32,u32)> rather than setting a field on App. The caller decides when to issue the resize. drain_messages() propagates the value upward. This keeps window management logic out of the App struct.

8. Caveats & Known Limitations

Startup white flash (minimised, not eliminated)

DWM fills every new window with white before the application paints. Mitigated by hiding the window until the image is ready and painting gray synchronously before the event loop starts. In normal use the white background is never visible.

No status bar

Filename, image index, and dimensions are not displayed. Showing text requires a font rasteriser crate, which would add significant binary size.

Animated GIF only

Animation is supported for GIF. APNG and animated WebP or AVIF would need separate decoder paths.

Render loop is O(win_w × win_h)

The renderer visits every window pixel on every redraw. For typical window sizes on modern hardware this completes well within a frame budget. The loop uses simple counter arithmetic — no division or modulo per pixel.

JXL HDR clamped, not tone-mapped

JPEG XL supports HDR content with pixel values above 1.0. f32_to_u8 clamps these to 255, losing highlight detail. A tone-mapping operator would produce better results for HDR files, but standard JXL photos are unaffected.

SVG external resources not loaded

usvg with default options does not resolve external fonts or linked images. SVGs that depend on external resources will render with missing content.

Work area heuristic on non-Windows

The precise SPI_GETWORKAREA path is Windows-only. Other platforms use 90% × 85% of the monitor size, which may not account for taskbars or docks accurately.

9. Installer — main.wxs

Built with WiX Toolset 3 via cargo-wix. Run cargo wix from the project root to produce an MSI in target\wix\. Use cargo wix --nocapture to see full compiler and linker output when debugging.

9.1 Product identity

The **UpgradeCode** GUID is fixed and must never change — it is what Windows uses to identify that a new version is an upgrade of the same product. **Product Id='*'** generates a fresh GUID on every build, which is correct for major upgrades. **MajorUpgrade** ensures that installing a new version automatically removes the old one.

9.2 Codepage constraints

WiX 3 does not support UTF-8 (codepage 65001) in Product/@Codepage or Package/@SummaryCodepage. Both are set to 1252 and the manufacturer name is ASCII-transliterated ('Hizlikan'). WiX 4 handles UTF-8 correctly but cargo-wix only supports WiX 3.

9.3 File associations — explicit RegistryValue elements

WiX 3's built-in ProgId/Extension elements produce duplicate primary keys (LGHT0130) when registering multiple extensions under one ProgId. This is a known WiX 3 bug. The workaround is to write all registry entries as explicit RegistryValue elements, giving full control over the Registry table.

9.4 Three installer features

Feature	Default	What it does
Add to Open With	✓ On	Adds Devoid Viewer to the right-click Open With menu for all supported formats witho
Set as Default Image Viewer	✓ On	Makes Devoid Viewer the default opener for all supported formats, replacing the curre
Desktop Shortcut	✓ On	Places a shortcut to Devoid Viewer on the desktop.

9.5 Registry structure

- **ProgIdRegistration** — always installed. Writes HKCR\DevoidViewer.ImageFile (display name), shell\open\command (launch command with "%1"), and DefaultIcon. Both association features depend on this.
- **OpenWithAssociations** — writes HKCR\.ext\OpenWithProgIds\DevoidViewer.ImageFile for each of the 11 supported extensions. Adds to Open With without changing the existing default.
- **DefaultAssociations** — writes HKCR\.ext = DevoidViewer.ImageFile for each extension, making Devoid Viewer the default opener.

WiX removes all registry entries on uninstall automatically since they are owned by components that are uninstalled with the product.

9.6 Shell icon cache refresh

A CustomAction runs ie4uinit.exe -ClearIconCache after install and uninstall so Explorer shows the correct file type icons immediately. Execute='immediate' — no rollback requirement.

9.7 Supported extensions

jpg, jpeg, png, bmp, gif, webp, tiff, tif, ico, jxl, svg — matching the formats supported by the decoder.

10. Glossary of Rust Concepts Used

Ownership
Every value has exactly one owner. When the owner goes out of scope the value is dropped. No garbage collector needed.

Borrowing
Lend a reference (&T;) without transferring ownership. References cannot outlive the value they point to.

&mut; self vs &self;
&mut; self gives exclusive mutable access. &self; gives shared read-only access. The compiler enforces that both cannot coexist.

Option
Either Some(value) or None. The safe replacement for null pointers. The compiler requires every Option to be handled.

Result
Either Ok(value) or Err(error). The ? operator propagates errors automatically up the call stack.

Vec
A growable heap-allocated array. Used for pixel buffers, frame lists, and path lists.

HashMap
A hash table. O(1) average lookup, insert, and delete. Used for the preload cache keyed by PathBuf.

Arc
Atomically Reference Counted smart pointer. Allows shared ownership across threads. Used for the Window handle.

Enum with data
Rust enums carry data per variant. Msg::Loaded carries a PathBuf, a DecodedImage, and an OpenMode — unlike C enums which carry no data.

then_some
(condition).then_some(value) returns Some(value) if the condition is true, None otherwise. Used in apply_image() to conditionally return resize dimensions.

Lifetime
A compile-time label on a reference. View<'a> borrows from App — the compiler ensures View cannot outlive App.

Closure
An anonymous function. The move keyword transfers ownership of captured variables into the closure — required for event_loop.run() and with_msg_hook().

match
Exhaustive pattern matching. Every variant of an enum must be handled or the code will not compile.

#[derive(...)]
Auto-generates trait implementations. #[derive(Clone, Debug)] writes clone() and the debug formatter for free.

Send
Marker trait meaning a type is safe to transfer to another thread. Derived automatically for types whose fields are all Send.

ControlFlow
Winit enum controlling the event loop sleep behaviour. Wait sleeps until any event. WaitUntil sleeps until a specific deadline. Used for GIF frame timing.

EventLoopProxy
Thread-safe handle to the event loop. proxy.send_event() injects a synthetic event from any thread, waking the event loop if it is sleeping.

NonZeroU32
A u32 guaranteed to be non-zero at compile time. softbuffer's resize method requires this to prevent zero-size surface creation.

cfg(windows)
Conditional compilation attribute. Code inside #[cfg(windows)] blocks is compiled only on Windows, keeping the binary clean on other platforms.

unsafe
Opts into calling C API functions where the compiler cannot verify safety. Used for MessageBoxW, SystemParametersInfoW, and PostMessageW.

type alias
type Proxy = ...; creates a shorter name with no runtime cost. Used to avoid repeating the verbose EventLoopProxy type.

Zero-size struct
struct WakeUp; has no fields and occupies zero bytes at runtime. Used purely as a type-level marker to wake the event loop.

include_bytes!

Embeds a file's raw bytes into the binary at compile time. Used for devoid.ico so the icon is always available regardless of the working directory at runtime.

msg_hook

winit 0.29 Windows platform extension. A callback that receives every Win32 message before winit processes it. Used to intercept WM_SYSCOMMAND/SC_CLOSE and re-post it as WM_CLOSE.